



密码学引论实验报告

姓名: 江李阳, 王健一, 邱泽辉

学号: 202400460042, 202400460037, 202400460046

专业: 密码科学与技术

日期: May 13, 2026

Contents

1	实验目的	2
2	实验环境	2
3	实验原理	3
3.1	CBC 模式解密关系	3
3.2	PKCS#7 填充规则	3
3.3	Padding Oracle 的泄露点	4
3.4	利用填充反馈恢复中间状态	4
3.5	攻击本质	5
4	目标密文与分组	5
5	攻击算法设计	5
6	代码实现说明	5
7	运行结果与破解明文	5
8	实验分析与总结	5

1 实验目的

本次实验的目的是理解并实现针对 CBC 工作模式下 PKCS#7 填充机制的 Padding Oracle 攻击。通过实验，掌握在未知 AES 密钥的情况下，如何利用解密服务器对填充是否合法的反馈信息，逐字节恢复密文对应的明文。

具体而言，本实验需要完成以下目标：

1. 理解 AES-128-CBC 模式的加解密结构，明确明文分组、密文分组和初始向量 IV 之间的关系；
2. 掌握 PKCS#7 填充规则，理解填充合法性判断如何泄露明文相关信息；
3. 编写 Padding Oracle 攻击程序，利用服务器返回的填充状态恢复 CBC 解密过程中的中间状态；
4. 对本小组抽到的 5 号密文进行破解，得到完整明文；
5. 通过实验分析 CBC 模式在缺少完整性保护时可能产生的安全风险，加深对“加密不等于认证”的理解。

2 实验环境

本实验在本地编写攻击程序，并通过访问实验指导中给出的解密服务器完成 Padding Oracle 攻击。实验过程中，攻击者不需要知道 AES 密钥，只需要根据服务器返回的填充状态判断密文解密后的 PKCS#7 填充是否合法。

Table 1: 实验环境

项目	配置或说明
操作系统	Windows 11
编程语言	Python 3
开发工具	Visual Studio Code
加密算法	AES-128-CBC
填充标准	PKCS#7
目标密文编号	5 号密文
密钥组	Key_2
解密接口	http://10.102.33.67:8208/dec_2?data=

其中，实验指导给出的解密服务器会对输入的密文进行 AES-128-CBC 解密，并检查解密后明文是否满足 PKCS#7 填充规则。若填充正确，服务器返回 HTTP 200；若填充错误，服务器返回 HTTP 500 server error。因此，本实验程序通过构造不同的密文并观察服务器返回状态，逐步恢复目标密文对应的明文。

由于本小组抽到的是 5 号密文，而实验材料中 5 号密文属于 Key_2 加密得到的密文，因此程序中访问的解密接口为 dec_2。实验运行时需要保证本机能够访问学校实验服务器，通常需要处于校园网环境或连接学校 VPN。

3 实验原理

本实验的核心原理是 **Padding Oracle** 攻击。该攻击针对采用 **CBC** 工作模式并使用 **PKCS#7** 填充的分组密码系统。当解密服务器会向攻击者反馈“填充是否合法”时，即使攻击者不知道 **AES** 密钥，也可以通过构造密文并反复查询服务器，逐字节恢复目标密文对应的明文。

3.1 CBC 模式解密关系

在 **CBC** 模式中，设初始向量为 IV ，密文分组为 $C_1, C_2, \dots, C_n$ ，对应明文分组为 $P_1, P_2, \dots, P_n$ 。令 $C_0 = IV$ ，则第 i 个密文分组的 **CBC** 解密关系为：

$$P_i = D_K(C_i) \oplus C_{i-1} \quad (1)$$

其中， $D_K(C_i)$ 表示在密钥 K 下对密文分组 C_i 进行 **AES** 解密得到的中间状态， $\oplus$ 表示按字节异或运算。

为了便于描述，记：

$$I_i = D_K(C_i) \quad (2)$$

则有：

$$P_i = I_i \oplus C_{i-1} \quad (3)$$

因此，如果能够恢复出中间状态 I_i ，再与真实的前一密文分组 C_{i-1} 异或，就可以得到真实明文分组 P_i 。

3.2 PKCS#7 填充规则

AES 的分组长度为 16 字节，而明文长度不一定正好是 16 字节的整数倍，因此需要使用填充机制。**PKCS#7** 填充规则规定：如果需要填充 t 个字节，则每个填充字节的值都为 t 。

例如，当需要填充 1 个字节时，填充值为：

$$01 \quad (4)$$

当需要填充 2 个字节时，填充值为：

$$02 \ 02 \quad (5)$$

当需要填充 6 个字节时，填充值为：

$$06 \ 06 \ 06 \ 06 \ 06 \ 06 \quad (6)$$

因此，解密服务器在解密后会检查最后若干字节是否符合 **PKCS#7** 填充规则。如果填充合法，则返回正确信息；如果填充不合法，则返回错误信息。

3.3 Padding Oracle 的泄露点

在本实验中，攻击者可以访问解密服务器。服务器对输入密文进行 AES-128-CBC 解密后，会检查解密结果是否满足 PKCS#7 填充规则：

- 若填充正确，服务器返回 HTTP 200；
- 若填充错误，服务器返回 HTTP 500 server error。

从表面上看，服务器没有直接返回明文或密钥。但它返回的填充状态本身已经泄露了关于解密结果的信息。攻击者可以不断构造不同的密文输入，根据服务器返回的 HTTP 状态码判断某种猜测是否正确。这种能够判断填充是否合法的服务器就称为 Padding Oracle。

3.4 利用填充反馈恢复中间状态

假设攻击目标是某个密文分组 C_i 。攻击者构造一个伪造的前一分组 R ，并向服务器提交：

$$R \parallel C_i \quad (7)$$

服务器解密最后一个分组后得到：

$$D_K(C_i) \oplus R = I_i \oplus R \quad (8)$$

由于 R 是攻击者可以控制的，攻击者就可以通过修改 R 的字节，使得 $I_i \oplus R$ 的最后若干字节变成合法的 PKCS#7 填充。

例如，为了恢复最后一个字节，攻击者枚举 R 的最后一个字节 R_{15} ，直到服务器返回 HTTP 200。此时说明：

$$I_{i,15} \oplus R_{15} = 01 \quad (9)$$

于是可以得到：

$$I_{i,15} = R_{15} \oplus 01 \quad (10)$$

恢复出最后一个中间状态字节后，攻击者继续构造伪造分组，使最后两个字节变成：

$$02 \ 02 \quad (11)$$

然后枚举倒数第二个字节，恢复 $I_{i,14}$ 。依次类推，可以从后向前逐字节恢复整个中间状态：

$$I_i = D_K(C_i) \quad (12)$$

最后，根据 CBC 解密关系：

$$P_i = I_i \oplus C_{i-1} \quad (13)$$

即可得到目标密文分组 C_i 对应的真实明文分组。

3.5 攻击本质

Padding Oracle 攻击并不是直接破解 AES 算法本身，也不是恢复 AES 密钥，而是利用了解密系统实现中的信息泄露。具体来说，服务器把“填充正确”和“填充错误”以不同的状态返回给攻击者，使攻击者能够通过大量选择密文查询，逐步推断 CBC 解密的中间状态，最终恢复明文。

因此，本实验说明：只提供机密性而缺少完整性保护的 CBC 加密实现是不安全的。在实际系统中，不能将可区分的 padding 错误直接暴露给外部用户，更安全的做法是使用认证加密模式，或者在解密前先进行消息认证。

4 目标密文与分组

5 攻击算法设计

6 代码实现说明

7 运行结果与破解明文

8 实验分析与总结